

Evaluación del módulo #4

Infraestructura Viva para

"Soluciones Digitales ACME"

Introducción

El presente portafolio desarrolla la propuesta “Infraestructura Viva”, orientada a modernizar la infraestructura tecnológica de Soluciones Digitales ACME mediante una migración progresiva desde un entorno on-premise hacia una arquitectura basada en servicios cloud.

Actualmente los departamentos de ventas, soporte y finanzas de ACME utilizan un entorno on-premise que presenta las siguientes limitaciones:

- Almacenamiento físico limitado y costoso: Restricciones de capacidad que frenan el crecimiento operacional.
- Sin diferenciación de datos activos vs. Históricos: Todos los datos se almacenan al mismo costo, sin importar su frecuencia de uso.
- Ausencia de respaldo automático de la Base de Datos: Riesgo de pérdida de información transaccional ante fallos de hardware.
- Mantenimiento manual de servidores: Alto costo en tiempo y personal para mantener la infraestructura operativa.
- Sin política de retención ni expiración: Acumulación de datos obsoletos que consumen espacio y dificultan auditorías.

Para resolver este problema, se abordan los principales componentes vistos durante el módulo 4: almacenamiento en la nube, bases de datos relacionales y NoSQL, servicios de cómputo, red virtual, alojamiento web, monitoreo, notificaciones y mensajería.

El desarrollo práctico se realizó utilizando Floci como entorno local de simulación de servicios AWS, permitiendo implementar y probar componentes cloud sin generar costos reales en una cuenta AWS, validando configuraciones como buckets de almacenamiento, bases de datos, colas de mensajería, servicios de notificación y otros elementos representativos de una arquitectura cloud. De esta forma, el proyecto combina el diseño conceptual con una implementación práctica, segura y de bajo costo, alineada con los requerimientos del caso.

Solución Propuesta

Propuesta Almacenamiento

Soluciones Digitales ACME opera actualmente con una infraestructura de almacenamiento on-premise que genera altos costos de mantenimiento, limita la escalabilidad y no cuenta con mecanismos automáticos de respaldo. Esta propuesta presenta la implementación de la primera etapa del proyecto "Infraestructura Viva": la migración de la capa de almacenamiento hacia la nube.

Para esto se propone:

- Implementar tres buckets Amazon S3 con roles diferenciados: almacenamiento de archivos estáticos activos, archivado histórico de largo plazo, y respaldo automático de la base de datos transaccional acme-postgres.
- Configurar políticas de ciclo de vida que muevan los datos automáticamente entre tiers de almacenamiento (Standard → Standard-IA → Glacier), reduciendo costos operacionales sin intervención manual.
- Establecer una estrategia de backup diario automatizable que proteja la información transaccional de los departamentos de ventas, soporte y finanzas.

Beneficios clave para la dirección

- Reducción de costos: Los datos históricos migran automáticamente a Glacier, reduciendo hasta un 75% el costo de almacenamiento frente a mantener todo en Standard.
- Protección de datos: Backups diarios automatizados de la base transaccional, con retención de 90 días y archivado en Glacier.
- Cero mantenimiento físico: Eliminación de hardware de almacenamiento on-premise; toda la gestión es gestionada en la nube.
- Escalabilidad inmediata: S3 escala automáticamente sin necesidad de planificar capacidad ni adquirir nuevo hardware.
- Cumplimiento normativo: Retención controlada de datos históricos con expiración automática, facilitando auditorías.

La solución se implementa sobre Amazon S3 y S3 Glacier emulados en Floci, organizando el almacenamiento en tres buckets con responsabilidades específicas y políticas de ciclo de vida automáticas.

Propuesta Bases de datos relacionales

Soluciones Digitales ACME utiliza datos transaccionales para áreas como ventas, soporte y finanzas. Mantener esta base de datos en un entorno on-premise implica administrar servidores físicos, respaldos, disponibilidad, capacidad y mantenimiento de infraestructura,

lo que aumenta costos operativos y limita la velocidad para crecer o probar nuevas funcionalidades.

Se propone migrar la base de datos relacional a un servicio gestionado tipo Amazon RDS, usando PostgreSQL como motor principal. PostgreSQL permite trabajar con información estructurada, relaciones entre tablas y consultas SQL, por lo que es adecuado para registrar clientes, tickets, incidentes y otros datos operacionales de ACME.

Para la primera etapa se plantea una instancia ``acme-postgres`` con base de datos ``acme``, motor PostgreSQL 16, clase ``db.t3.micro``, 20 GB de almacenamiento, backups automáticos por 7 días y configuración Multi-AZ como referencia de alta disponibilidad. En la demo local, Floci permite simular esta configuración sin generar costos reales en AWS.

La migración se justifica por estos puntos:

- Reducir la dependencia de hardware físico y tareas manuales de mantenimiento.
- Facilitar respaldos y recuperación mediante backups automáticos y respaldos manuales con ``pg_dump``.
- Mejorar la disponibilidad usando una arquitectura gestionada y preparada para alta disponibilidad.
- Permitir crecimiento vertical de CPU, memoria o almacenamiento según demanda.
- Mantener compatibilidad con SQL y con un modelo relacional simple para las áreas internas de ACME.
- Separar la base de datos de la infraestructura local, dejando una base preparada para integrarse con otros servicios cloud.

El prototipo valida la propuesta creando la instancia, conectando con ``psql``, creando tablas de prueba ("clientes" y "tickets"), ejecutando consultas SQL y generando un respaldo manual.

Propuesta Bases de datos NoSQL

Se propone incorporar Amazon DynamoDB como motor NoSQL para gestionar los tickets de soporte de ACME. DynamoDB es un servicio serverless y completamente gestionado dentro del ecosistema AWS, lo que elimina la necesidad de administrar servidores, configurar réplicas o planificar capacidad manualmente. En la demo local, Floci permite simular esta configuración sin generar costos reales en AWS.

Para la primera etapa se crea la tabla **TicketsSoporte** con clave primaria **ticketId** en formato **TK-AAAA-NNNN**, modo de facturación **PAY_PER_REQUEST** y tres índices secundarios globales: **estado-index** para filtrar tickets por su estado actual, **tecnico-index** para consultar la carga de trabajo de cada técnico, y **prioridad-index** para identificar incidentes críticos de forma inmediata. El campo **clienteId** en cada ticket referencia los registros de clientes almacenados en RDS, conectando ambas bases de datos sin duplicar información.

La incorporación de DynamoDB se justifica por estos puntos:

- Manejar datos semiestructurados donde cada ticket puede tener atributos variables según el tipo de incidente, sin afectar la integridad del resto de los registros.
- Complementar la base relacional RDS separando claramente los datos transaccionales de los datos operacionales semiestructurados.
- Facilitar respaldos exportando los datos a archivos JSON que pueden almacenarse en S3 para historial y auditoría.

El prototipo valida la propuesta creando la tabla con sus tres índices secundarios, insertando tickets representativos de los departamentos de ventas, soporte y finanzas, y ejecutando consultas por cada índice para demostrar que la solución responde correctamente a los requerimientos operacionales de ACME.

Propuesta servicios de cómputo

Se desplegará una aplicación web sencilla en AWS utilizando un servicio de cómputo escalable, con el fin de comprender el proceso de implementación, monitorear su funcionamiento y evaluar su comportamiento ante un incremento en la carga de trabajo.

Servicio de cómputo seleccionado

Se propone utilizar “Amazon EC2” debido a que permite crear máquinas virtuales en la nube de forma rápida y sencilla. Además, está disponible dentro del programa AWS Academy y del Free Tier, lo que facilita su implementación sin costos adicionales.

Descripción de la propuesta

La propuesta consiste en desplegar una aplicación web estática (HTML, CSS y JavaScript) en una instancia EC2 con un servidor web Apache instalado. La aplicación será accesible mediante la dirección IP pública de la instancia.

La infraestructura estará conformada por:

- Una instancia Amazon EC2.
- Una red virtual (VPC) con acceso a Internet.
- Un Security Group que permita conexiones por HTTP (puerto 80) y SSH (puerto 22).
- Un servidor Apache para alojar la aplicación.
- Amazon CloudWatch para monitorear el rendimiento de la instancia.

Metodología de implementación

1. Crear una instancia Amazon EC2 utilizando Amazon Linux.
2. Configurar el grupo de seguridad habilitando los puertos necesarios.
3. Conectarse mediante SSH.
4. Instalar el servidor web Apache.
5. Copiar la aplicación web al directorio del servidor.

6. Verificar el acceso desde un navegador utilizando la IP pública.
7. Supervisar el rendimiento mediante Amazon CloudWatch.

Prueba de escalabilidad

Para evaluar el comportamiento de la aplicación, se realizará una prueba de carga mediante múltiples solicitudes simultáneas utilizando herramientas como Apache Benchmark (ab) o varias conexiones concurrentes desde el navegador.

Durante la prueba se observarán las siguientes métricas:

- Utilización de CPU.
- Uso de memoria.
- Tiempo de respuesta.
- Disponibilidad del servicio.

Resultados esperados

Se espera que la aplicación permanezca disponible durante la prueba, respondiendo correctamente a las solicitudes de los usuarios. Asimismo, se espera observar un incremento temporal en el uso de los recursos durante la carga y una disminución una vez finalizada la prueba.

Recursos necesarios

- Cuenta AWS Academy o AWS Free Tier.
- Amazon EC2.
- Amazon CloudWatch.
- Servidor Apache.
- Aplicación web estática.

La práctica permitió comprender el proceso de despliegue de aplicaciones utilizando Amazon EC2 y comprobar el funcionamiento de un servicio de cómputo en la nube. Asimismo, se observó cómo AWS facilita el monitoreo del rendimiento mediante CloudWatch y proporciona la base para implementar soluciones escalables que pueden crecer de acuerdo con la demanda de los usuarios.

Propuesta de servicio de red en la nube

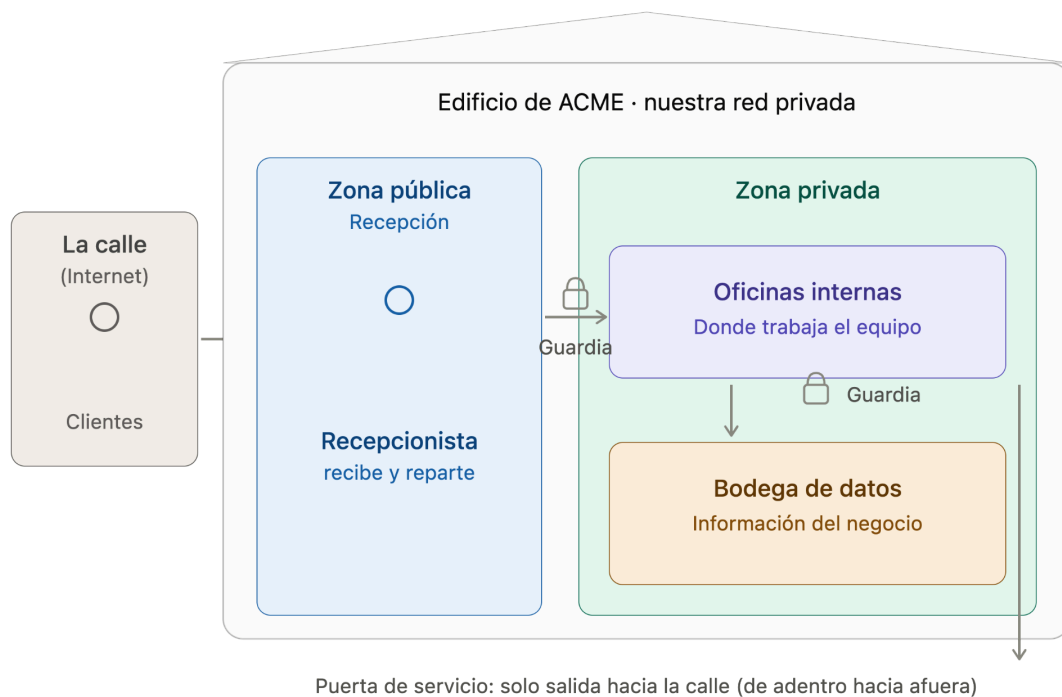


Figura 1: Diagrama arquitectura de red para acme.

Imaginemos que ACME alquila un edificio propio y cercado en la nube. Ese edificio es nuestra red privada: nadie de afuera entra a menos que lo dejemos pasar por la puerta correcta.

El edificio tiene dos zonas. Una recepción de cara a la calle, que es la única parte que el público puede ver, y unas oficinas internas al fondo, donde está lo valioso —los empleados trabajando y los archivadores con la información del negocio— a las que el público jamás llega directamente.

Cuando un cliente entra por la puerta, no camina por donde quiere: lo recibe un recepcionista que lo atiende y lo dirige. Si mañana llegan diez veces más clientes, ponemos más recepcionistas y la atención no se cae. Ese recepcionista es el único expuesto al exterior; protege todo lo de atrás.

Para que nadie se cuele, hay guardias en cada puerta: a las oficinas internas solo se entra si vienes **derivado** por recepción, y los archivadores con los datos del negocio **solo los abre el personal autorizado**. Doble control, por si un guardia falla.

Los empleados, cuando necesitan algo de la calle (una actualización, un proveedor), salen por una puerta de servicio que solo abre hacia afuera: ellos pueden salir, pero nadie entra por ahí. Y para llegar a la bodega de archivos pesados (fotos, documentos, históricos) usan un pasillo interno privado, sin tener que pisar la calle, lo cual es más seguro y además no nos cuesta nada.

Eso es todo. En una frase: construimos un edificio donde lo público y lo privado están separados, con recepción y guardias, de modo que el negocio escala, está protegido y gasta lo justo.

Para información técnica sobre servicios, tecnología e implementación, revisar el capítulo de documentación técnica sobre Servicio de Red en la Nube.

Notificación y Mensajería

Soluciones Digitales ACME necesita un mecanismo confiable para comunicar eventos entre servicios y alertar al equipo de soporte ante incidentes. En un entorno on-premise, este flujo suele depender de integraciones directas entre aplicaciones, lo que aumenta el acoplamiento, dificulta el manejo de fallos y puede provocar pérdida de mensajes si un servicio no está disponible.

Se propone implementar una solución de mensajería basada en servicios gestionados tipo Amazon SNS y Amazon SQS. SNS funcionará como canal central de publicación de alertas mediante el tema `acme-alertas-topic`, mientras que SQS recibirá los mensajes en la cola `acme-alertas-queue` para que puedan ser procesados de forma asíncrona. Esto permite desacoplar el servicio que genera la alerta del servicio que la consume.

Para mejorar la tolerancia a fallos, se incorpora una Dead Letter Queue llamada `acme-alertas-dlq`. Esta cola almacenará mensajes que no puedan procesarse correctamente después de varios intentos. La política `RedrivePolicy` se configura con `maxReceiveCount` igual a 3, permitiendo demostrar un patrón básico de reintentos y revisión posterior de errores.

La migración se justifica por estos puntos:

- Reducir el acoplamiento entre aplicaciones y servicios internos.
- Evitar pérdida de mensajes cuando un consumidor no esté disponible.
- Permitir procesamiento asíncrono de alertas e incidentes.
- Mejorar la resiliencia mediante reintentos y DLQ.
- Facilitar futuras integraciones con correo, SMS, Lambda o endpoints HTTP/S.
- Centralizar la publicación de eventos críticos de infraestructura.
- Preparar la arquitectura para monitoreo y respuesta automática ante incidentes.

En la demo local, Floci permite simular SNS y SQS sin generar costos reales en AWS. El prototipo valida la propuesta creando el tema SNS, la cola SQS principal, la DLQ, la suscripción entre SNS y SQS, una alerta simulada de CPU alta, la lectura del mensaje desde SQS y la política de reintentos hacia la DLQ.

Servicios simples de alojamiento web y contenido

En esta etapa, se desea poder publicar contenido estático y distribuirlo globalmente con

buena performance. Para lo anterior, se propone crear un bucket S3 que albergue el sitio web (archivo html) donde se debe aplicar los certificados y configuraciones de seguridad.

Métricas a revisar:

- Un sitio web público en S3 (estático)
- Distribución CloudFront implementada
- 2 configuraciones de seguridad

Diagrama de arquitectura propuesto:

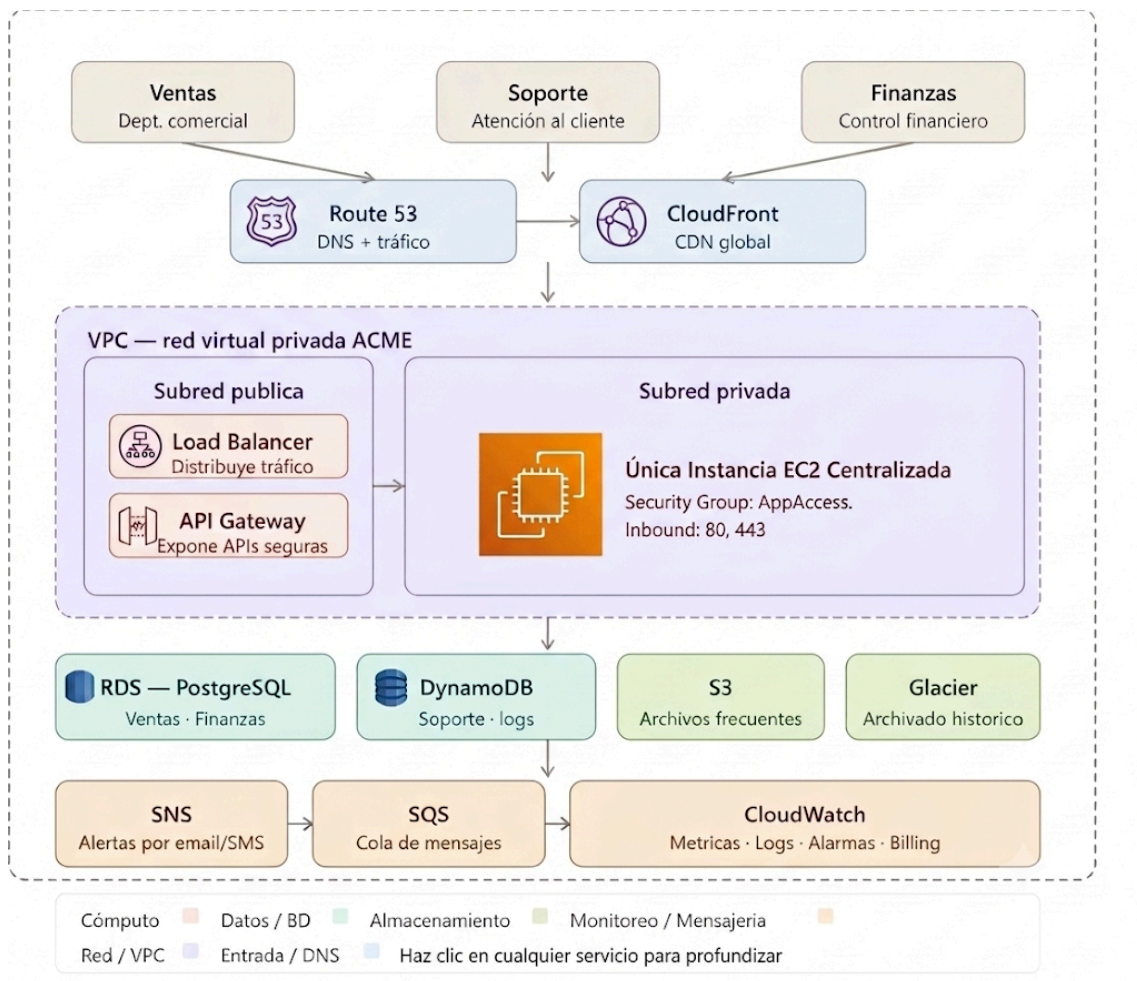


Figura 2: Diagrama de arquitectura propuesto para Acme.

Explicación del diagrama:

1. Cómputo escalable y seguro: Los tres departamentos acceden vía Route 53 y CloudFront. Dentro de la VPC, la subred privada aloja EC2 (aplicaciones estables). Un Load Balancer (balanceador) distribuye la carga entre ellos.

2. Bases de datos sin hardware físico: RDS con PostgreSQL gestiona los datos estructurados de ventas y finanzas (transacciones, facturas, clientes). DynamoDB maneja

los datos de soporte, donde la flexibilidad NoSQL permite guardar tickets, logs y conversaciones en formato libre.

3. Almacenamiento diferenciado: S3 guarda documentos, contratos e imágenes de acceso frecuente. Glacier archiva datos históricos (facturas antiguas, registros de auditoría) a bajo costo, con recuperación en horas cuando se necesita.

4. Red virtual aislada: La VPC divide el tráfico en dos subredes: la pública expone solo el Load Balancer y el API Gateway (lo que debe ser accesible desde internet), mientras la subred privada protege todo el cómputo y los datos. Los Security Groups actúan como firewall por servicio.

5. Monitoreo proactivo: CloudWatch recolecta métricas de CPU, memoria y errores en tiempo real. Cuando algo supera un umbral (por ejemplo, un pico de carga en ventas), dispara una alarma que SNS convierte en notificación por email o SMS al equipo responsable. SQS además desacopla los procesos pesados para que no bloqueen el sistema.

6. Solución con los conceptos del manual: Cada servicio del diagrama corresponde directamente a los contenidos vistos: VPC, EC2, RDS, DynamoDB, S3, Glacier, Route 53, CloudFront, SNS, SQS y CloudWatch.

Diagrama de transición

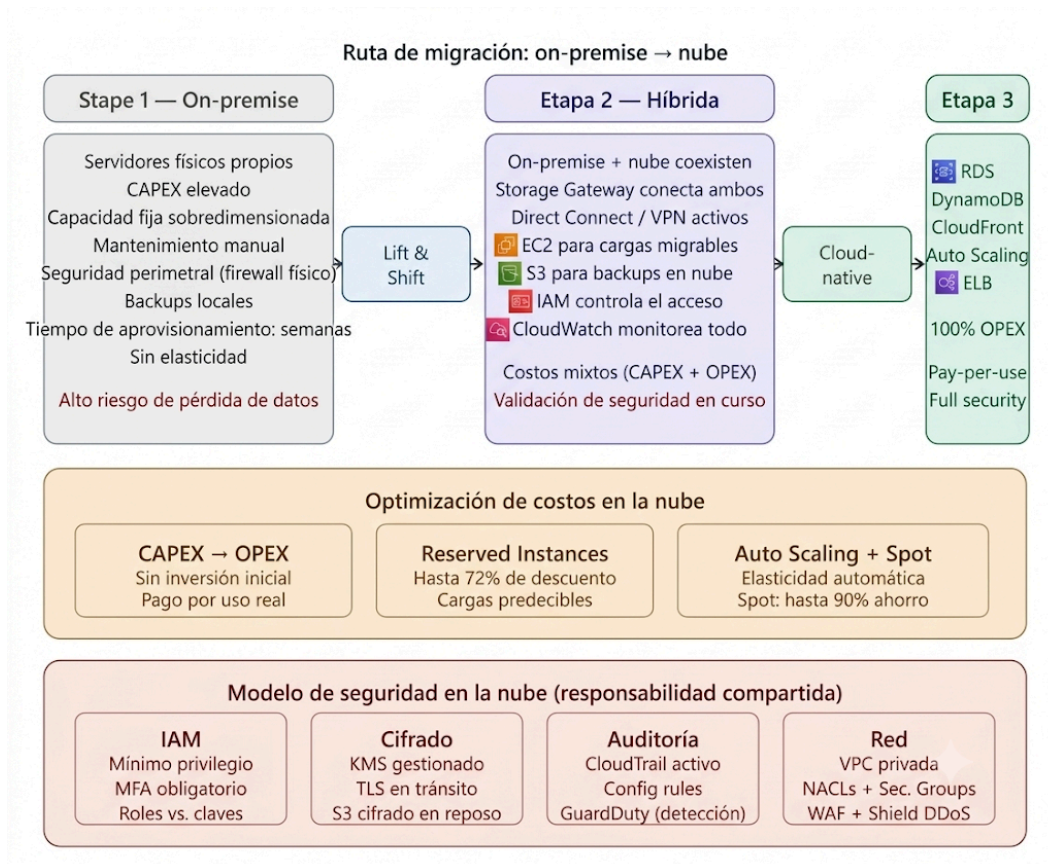


Figura 3: Diagrama de migración para acme.

Documentación Técnica

Almacenamiento

En esta primera lección del proyecto "Infraestructura Viva" se implementa la capa de almacenamiento utilizando Amazon S3 y S3 Glacier, emulados en el entorno local Floci, permitiendo:

- Eliminar la dependencia de hardware físico para almacenamiento de archivos estáticos.
- Implementar un sistema de archivado automático para datos históricos.
- Establecer una estrategia de respaldo confiable y automatizable para la base de datos acme-postgres.
- Optimizar costos diferenciando datos según su frecuencia de acceso.

Política de Ciclo de Vida

La solución contempla tres buckets S3 con roles específicos, cubriendo las métricas mínimas requeridas (1 bucket estándar + 1 archivado) y alcanzando el máximo permitido (3 buckets con políticas diferentes).

Bucket	Tipo S3	Propósito	Destino final
acme-static-files	S3 Standard	Archivos estáticos (imágenes, CSS, JS, PDFs)	Glacier (día 90)
acme-archive	S3 → Glacier	Datos históricos (reportes, logs, registros antiguos)	Glacier (día 1)
acme-db-backups	S3 Standard-IA	Respaldos automáticos de acme-postgres	Glacier (día 30)

Tabla 1: Resumen de los tres buckets S3 configurados en Floci.

Bucket	Día 0	Transición 1	Transición 2	Expiración
acme-static-files	Standard	Día 30 → IA	Día 90 → Glacier	Día 365
acme-archive	Standard	Día 1 → Glacier	—	Día 730
acme-db-backups	Standard	Día 7 → IA	Día 30 → Glacier	Día 90

Tabla 2: Resumen de transiciones de ciclo de vida por bucket.

Implementación en Floci

La implementación se realizó mediante scripts de AWS CLI apuntando al endpoint local de Floci (<http://localhost:4566>). Se desarrollaron tres scripts organizados en una carpeta de proyecto, más un script maestro que los ejecuta en secuencia.

Estructura de archivos del proyecto

acme-storage/ scripts/

script.sh	← Script maestro
01_crear_buckets.sh	← Crea los 3 buckets S3
02_lifecycle_policies.sh	← Configura ciclos de vida
03_backup_postgres.sh	← Backup de acme-postgres a S3

```
> data
$ 01_crear_buckets.sh
$ 02_lifecycle_policies.sh
$ 03_backup_postgres.sh
👉 docker-compose.yml
$ script.sh
```

crear_buckets.sh: Este script crea los tres buckets y les asigna etiquetas (tags) para facilitar la gestión de recursos por proyecto, área y empresa.

02_lifecycle_policies.sh: Cada bucket recibe una política de lifecycle diferente según el rol que cumple. Las políticas automatizan el movimiento de datos entre tiers de almacenamiento sin intervención manual.

03_backup_postgres.sh: Este script representa la integración con la base de datos acme-postgres ya existente en Floci. Genera un dump completo de la base acme, lo comprime y lo sube al bucket acme-db-backups con una estructura de carpetas por fecha.

Creación de Buckets

```
gabyd@Gaby MINGW64 /c/Floci
$ docker compose up -d
[+] up 2/2
✓ Network floci_default Created
✓ Container floci-floci-1 Started

gabyd@Gaby MINGW64 /c/Floci
$ chmod + script.sh

gabyd@Gaby MINGW64 /c/Floci
$ ./script.sh
```

Figura 4: comandos para iniciar contenedor para Floci.

```
gabyd@Gaby MINGW64 /c/Floci
$ ./script.sh
>>> PASO 1: Creando buckets S3...

=====
ACME - Creación de Buckets S3 en Floci
=====

[1/3] Creando bucket de archivos estáticos...
make_bucket: acme-static-files
✓ Bucket 'acme-static-files' creado (S3 Standard)

[2/3] Creando bucket de archivado...
make_bucket: acme-archive
✓ Bucket 'acme-archive' creado (destinado a Glacier)

[3/3] Creando bucket de backups de BD...
make_bucket: acme-db-backups
✓ Bucket 'acme-db-backups' creado (S3 Standard-IA)

=====
Buckets creados exitosamente:
=====
2026-06-29 18:20:15 acme-archive
2026-06-29 18:20:18 acme-db-backups
2026-06-29 18:20:12 acme-static-files
```

Figura 5: pasos de script para crear buckets en S3.

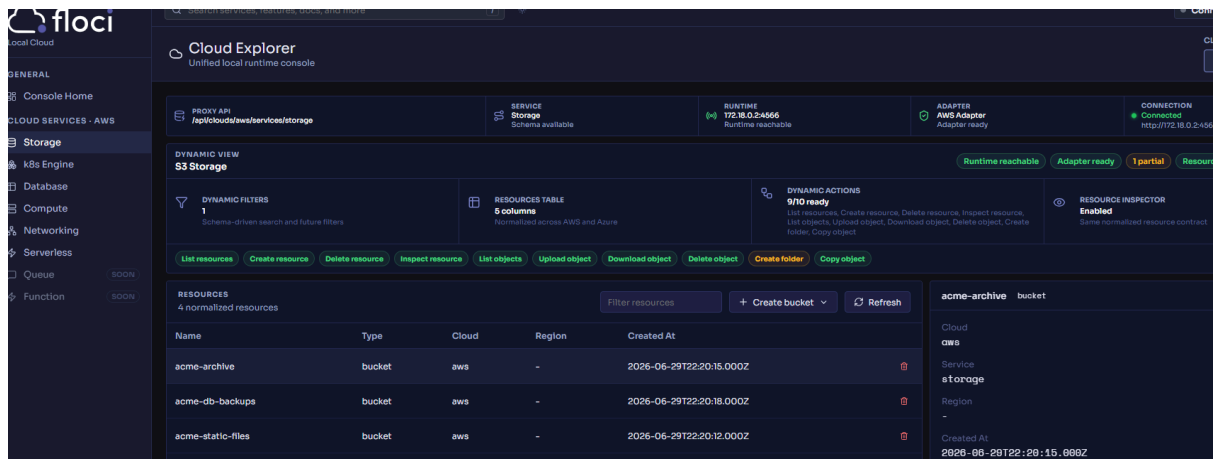


Figura 6: Buckets creados en Flocci.

Justificación del Tipo de Almacenamiento

La elección de los tiers de almacenamiento se basó en tres criterios principales: frecuencia de acceso esperada, costo relativo y disponibilidad requerida por el negocio.

1. S3 Standard — Acceso frecuente

- Uso: Archivos estáticos activos (imágenes del sitio, documentos del portal de clientes, backups recientes).
- Costo: Mayor costo de almacenamiento por GB, pero sin costo adicional por acceso.
- Disponibilidad: 99.99%, ideal para archivos que deben estar disponibles en todo momento.
- Justificación ACME: Los departamentos de ventas y soporte requieren acceso inmediato a sus archivos operacionales.

2. S3 Standard-IA — Acceso infrecuente

- Uso: Archivos que se consultan menos de una vez al mes (backups de la semana anterior, reportes cerrados).
- Costo: Aproximadamente 40% más económico que Standard en almacenamiento; cobra por acceso.
- Disponibilidad: 99.9%, aceptable para datos que no son críticos en tiempo real.
- Justificación ACME: Sirve como etapa intermedia antes del archivado, reduciendo costos sin perder accesibilidad razonable.

3. S3 Glacier — Archivado a largo plazo

- Uso: Datos históricos, logs de más de 90 días, backups de base de datos que superan el mes.

- Costo: Hasta un 75% más económico que Standard. El acceso tarda minutos u horas.
- Disponibilidad: Recuperación en horas (Glacier Flexible Retrieval) o minutos (Instant Retrieval).
- Justificación ACME: Los datos históricos de ventas y tickets sólo se consultan para auditorías o análisis retrospectivos; Glacier cubre este caso de uso al menor costo posible.

Tier	Costo relativo	Acceso	Disponibilidad	Uso en ACME
S3 Standard	\$\$\$	Inmediato	99.99%	Estáticos activos, backup diario
S3 Standard-IA	\$\$	Inmediato*	99.9%	Etapa intermedia, backup semana
S3 Glacier	\$	Minutos/horas	99.99%	Histórico, logs, archivos +90 días

Tabla 3: Comparativa de tiers S3 según costo, acceso y disponibilidad.

(*) Standard-IA cobra una tarifa mínima por recuperación de datos.

Bases de datos relacionales

Para esta etapa del proyecto “Infraestructura Viva”, se implementó una base de datos relacional utilizando un entorno local basado en Floci, simulando el comportamiento de Amazon RDS. El objetivo fue representar una base de datos gestionada para almacenar información transaccional de la empresa Soluciones Digitales ACME, evitando el uso de infraestructura física y reduciendo costos de prueba.

Se eligió PostgreSQL, ya que es una alternativa ampliamente utilizada, compatible con datos estructurados y adecuada para manejar información transaccional. La instancia fue creada con el identificador `acme-postgres`, una clase tipo `db.t3.micro`, 20 GB de almacenamiento y una base llamada `acme`.

```
aws rds create-db-instance \
  --db-instance-identifier acme-postgres \
  --engine postgres \
  --engine-version 16 \
  --db-instance-class db.t3.micro \
  --allocated-storage 20 \
  --master-username acmeadmin \
  --master-user-password "AcmePass123!" \
  --db-name acme \
  --backup-retention-period 7 \
  --multi-az \
  --no-publicly-accessible
```

Figura 7: comando de creación de la instancia RDS.

La configuración incluyó una retención de backups de 7 días mediante el parámetro `backup-retention-period`. Además, se consideró la opción Multi-AZ como referencia de alta disponibilidad, entendiendo que en un entorno real de AWS esta configuración permite aumentar la resiliencia de la base de datos frente a fallos de infraestructura. En Floci, esta configuración se utiliza principalmente como simulación y evidencia de diseño.

```
jffarias@jffarias:~/ARQUITECTURACLOUD/portafolio3$ aws rds describe-db-instances \
  --query "DBInstances[?DBInstanceIdentifier=='acme-postgres']|[0].{ID:DBInstanceIdentifier,Estado:DBInstanceStatus,Motor:Engine,MultiAZ:MultiAZ,Backup:BackupRetentionPeriod,Endpoint:Endpoint.Address,Puerto:Endpoint.Port}" \
  --output table
```

Backup	Endpoint	Estado	ID	Motor	MultiAZ	Puerto
None	172.21.0.2	available	acme-postgres	postgres	True	7001

Figura 8: comando `describe-db-instances` mostrando estado, motor, backup y Multi-AZ.

Luego se obtuvo el endpoint de conexión de la base de datos y se realizó la conexión desde WSL2 utilizando el cliente `psql`. Esto permitió validar que la instancia relacional estaba disponible y lista para recibir operaciones SQL.

```
jffarias@jffarias:~/ARQUITECTURACLOUD/portafolio3$ export DB_HOST=localhost
jffarias@jffarias:~/ARQUITECTURACLOUD/portafolio3$ export DB_PORT=$(aws rds describe-db-instances \
  --query "DBInstances[?DBInstanceIdentifier=='acme-postgres']|[0].Endpoint.Port" \
  --output text)
jffarias@jffarias:~/ARQUITECTURACLOUD/portafolio3$ echo $DB_HOST
localhost
jffarias@jffarias:~/ARQUITECTURACLOUD/portafolio3$ echo $DB_PORT
7001
jffarias@jffarias:~/ARQUITECTURACLOUD/portafolio3$ psql "host=$DB_HOST port=$DB_PORT dbname=acme user=acmeadmin password=AcmePass123!"
psql (14.20 (Ubuntu 14.20-0ubuntu0.22.04.1), server 16.13)
WARNING: psql major version 14, server major version 16.
Some psql features might not work.
SSL connection (protocol: TLSv1.3, cipher: TLS_AES_256_GCM_SHA384, bits: 256, compression: off)
Type "help" for help.

acme=#
```

Figura 9: conexión exitosa con `psql`.

Para probar el funcionamiento de la base, se crearon dos tablas principales: clientes y tickets. La tabla clientes representa áreas internas de ACME, como ventas, soporte y finanzas. La tabla tickets permite registrar solicitudes o incidentes asociados a cada área, incluyendo estado, prioridad y fecha de creación.

```
acme=# CREATE TABLE clientes (
    id SERIAL PRIMARY KEY,
    nombre VARCHAR(100) NOT NULL,
    area VARCHAR(50) NOT NULL
);

CREATE TABLE tickets (
    id SERIAL PRIMARY KEY,
    cliente_id INT REFERENCES clientes(id),
    descripcion TEXT NOT NULL,
    estado VARCHAR(30) NOT NULL,
    prioridad VARCHAR(20) NOT NULL,
    fecha_creacion DATE NOT NULL
);

INSERT INTO clientes (nombre, area) VALUES
('Ventas ACME', 'Ventas'),
('Soporte ACME', 'Soporte'),
('Finanzas ACME', 'Finanzas');

INSERT INTO tickets (cliente_id, descripcion, estado, prioridad, fecha_creacion) VALUES
(1, 'Error en módulo de ventas', 'abierto', 'alta', '2026-06-01'),
(2, 'Consulta por acceso de usuario', 'cerrado', 'media', '2026-06-03'),
(3, 'Reporte financiero lento', 'abierto', 'alta', '2026-06-05');
```

Figura 10: creación de tablas e inserción de datos.

```
acme=# SELECT * FROM clientes;
SELECT * FROM tickets;
```

id	nombre	area
1	Ventas ACME	Ventas
2	Soporte ACME	Soporte
3	Finanzas ACME	Finanzas

(3 rows)

id	cliente_id	descripcion	estado	prioridad	fecha_creacion
1	1	Error en módulo de ventas	abierto	alta	2026-06-01
2	2	Consulta por acceso de usuario	cerrado	media	2026-06-03
3	3	Reporte financiero lento	abierto	alta	2026-06-05

(3 rows)

Figura 11: Selección de datos en tablas.

También se generó un respaldo manual utilizando pg_dump, guardando una copia de la base en un archivo .sql. Esta acción complementa la configuración de backups y permite demostrar una estrategia básica de recuperación ante pérdida de datos.

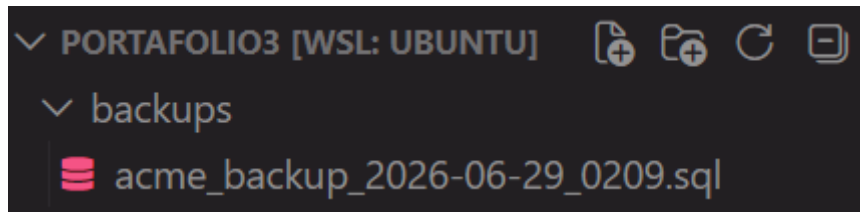


Figura 12: archivo de backup generado.

Además, se realizaron pruebas de consultas SQL en SQLiteOnline, utilizando una versión simplificada del modelo relacional. Se ejecutaron consultas para listar clientes, filtrar tickets abiertos, unir información entre tablas, agrupar tickets por prioridad y ordenar registros por fecha. Estas pruebas permitieron validar la lógica de consulta y demostrar el uso de SQL sobre datos estructurados.

```
acme=# SELECT c.nombre, c.area, t.descripcion, t.prioridad
FROM tickets t
JOIN clientes c ON t.cliente_id = c.id;
  nombre | area | descripcion | prioridad
-----+-----+-----+-----
 Ventas ACME | Ventas | Error en módulo de ventas | alta
 Soporte ACME | Soporte | Consulta por acceso de usuario | media
 Finanzas ACME | Finanzas | Reporte financiero lento | alta
(3 rows)
```

```
acme=# SELECT prioridad, COUNT(*) AS cantidad
FROM tickets
GROUP BY prioridad;
 prioridad | cantidad
-----+-----
 media | 1
 alta | 2
(2 rows)
```

```
acme=# SELECT *
FROM tickets
ORDER BY fecha_creacion DESC;
 id | cliente_id | descripcion | estado | prioridad | fecha_creacion
-----+-----+-----+-----+-----+-----
 3 | 3 | Reporte financiero lento | abierto | alta | 2026-06-05
 2 | 2 | Consulta por acceso de usuario | cerrado | media | 2026-06-03
 1 | 1 | Error en módulo de ventas | abierto | alta | 2026-06-01
(3 rows)
```

Figura 13: consultas SQL ejecutadas.

Respecto a costos, al utilizar Floci la implementación no genera cobros reales, ya que todo se ejecuta localmente en Docker. En un escenario real de AWS, los costos dependerían principalmente del tipo de instancia RDS, almacenamiento aprovisionado, backups, uso de Multi-AZ y transferencia de datos.

En cuanto a escalabilidad, la base relacional podría crecer verticalmente aumentando CPU, memoria o almacenamiento de la instancia. En una etapa más avanzada, también se podrían utilizar réplicas de lectura para descargar consultas y mejorar el rendimiento. De

esta forma, la solución propuesta permite partir con una configuración simple y de bajo costo, pero con posibilidad de evolucionar según las necesidades de ACME.

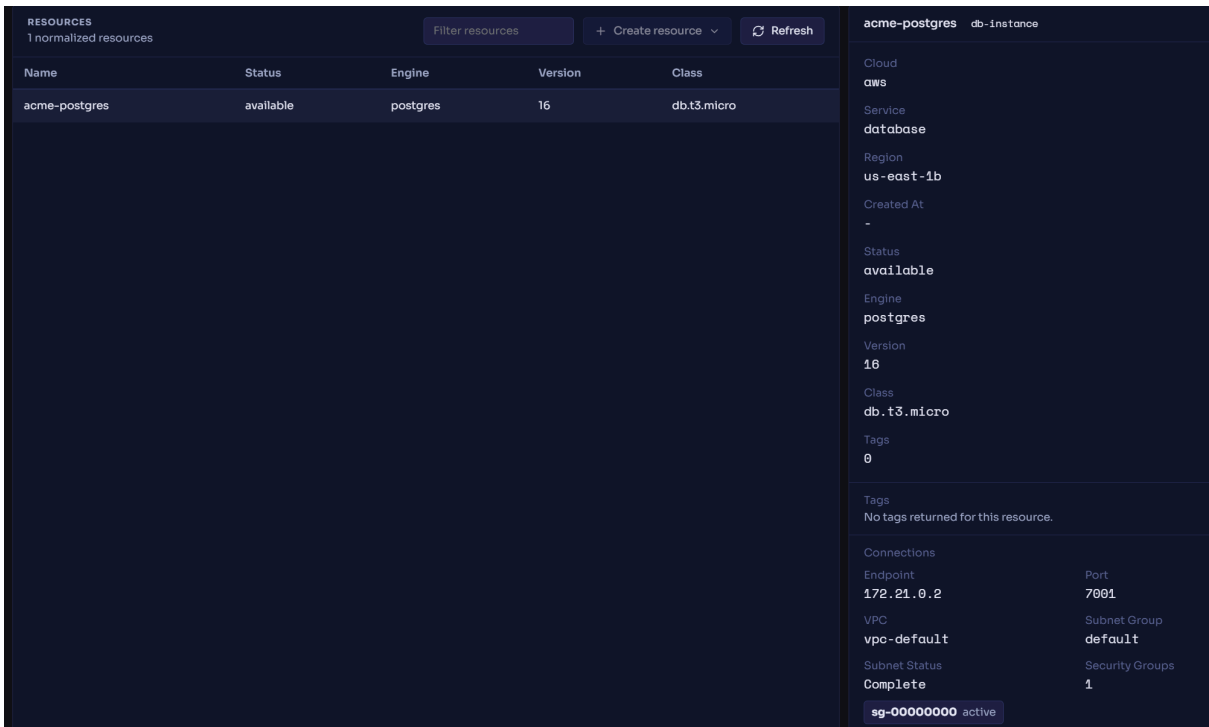


Figura 14: Configuración de base de datos en floci.

Bases de datos NoSQL

1. Objetivo

Desplegar e integrar una base de datos NoSQL para manejar datos semiestructurados, utilizando Amazon DynamoDB emulado localmente mediante Floci, AWS CLI y Docker Desktop desde Visual Studio Code.

2. Herramientas utilizadas

Herramienta	Versión / Detalle	Función
Floci (Local Cloud)	floci 0.1.8	Emula servicios AWS localmente, incluyendo DynamoDB
AWS CLI	v2.x	Ejecutar comandos DynamoDB desde PowerShell
Docker Desktop	29.5.3	Contenedor base para el runtime de Floci

3. Estructura del proyecto

la solución NoSQL se organiza en una carpeta dedicada con scripts numerados y archivos de datos independientes:

```
03_dynamodb/
├── gsi.json                # Definición de los 3 índices secundarios
├── 01_create_table.sh      # Crear la tabla TicketsSoporte
├── 02_check_table_state.sh # Verificar estado de la tabla
├── 03_insert_tickets.sh    # Insertar 5 tickets de ejemplo
├── 04_scan_table.sh        # Listar todos los tickets
├── 05_query_by_estado.sh   # Consulta por estado-index
├── 06_query_by_tecnico.sh   # Consulta por tecnico-index
├── 07_query_by_prioridad.sh # Consulta por prioridad-index
├── 08_create_backup.sh     # Generar respaldo en JSON
├── tk1.json ... tk5.json   # Datos de cada ticket
└── backups/                # Respaldos generados
```

4. Diseño de la tabla NoSQL

4.1 Nombre tabla y clave primaria

Tabla: TicketsSoporte

Clave primaria : ticketId (String) — Formato: TK-AAAA-NNNN

4.2 Atributos del esquema

Campo	Tipo	Ejemplo	Propósito
ticketId	String	TK-2026-0001	Identificador único del ticket
clienteId	String	CL-001	Referencia al cliente en RDS (base SQL)
departamento	String	Ventas	Área afectada por el incidente
categoria	String	Software	Clasifica el tipo de incidente
prioridad	String	Alta	Nivel de urgencia para atención
estado	String	Abierto	Estado actual del ticket
tecnico	String	Carlos Pérez	Técnico asignado para resolver
fechaCreacion	String	2026-06-28	Fecha de apertura del ticket
descripcion	String	Error al iniciar...	Descripción detallada del problema

5. Índices secundarios globales (GSI)

Se configuraron 3 índices secundarios globales para permitir consultas eficientes por diferentes dimensiones del negocio, sin necesidad de escanear toda la tabla.

Índice	Atributo clave	Caso de uso de negocio
estado-index	estado	El jefe de soporte consulta: "Muéstrame todos los tickets abiertos"
tecnico-index	tecnico	El supervisor consulta: "¿Cuántos tickets tiene asignados Carlos Pérez?"
prioridad-index	prioridad	El gerente consulta: "¿Hay tickets críticos sin atender?"

¿Por qué GSI?

Los Global Secondary Indexes (GSI) son más flexibles: permiten usar cualquier atributo como clave y son la única opción válida para consultar por estado, tecnico o prioridad, que son atributos distintos al ticketId.

6. Implementación paso a paso

6.1 Crear archivo de índices (gsi.json)

Se crea el archivo JSON con la definición de los 3 índices:

```
cat > gsi.json << 'EOF'
[
  {
    "IndexName": "estado-index",
    "KeySchema": [{"AttributeName": "estado", "KeyType": "HASH"}],
    "Projection": {"ProjectionType": "ALL"}
  },
  {
    "IndexName": "tecnico-index",
    "KeySchema": [{"AttributeName": "tecnico", "KeyType": "HASH"}],
    "Projection": {"ProjectionType": "ALL"}
  },
  {
    "IndexName": "prioridad-index",
    "KeySchema": [{"AttributeName": "prioridad", "KeyType": "HASH"}],
    "Projection": {"ProjectionType": "ALL"}
  }
]
EOF
```

6.2 Crear la tabla TicketsSoporte (01_create_table.sh)

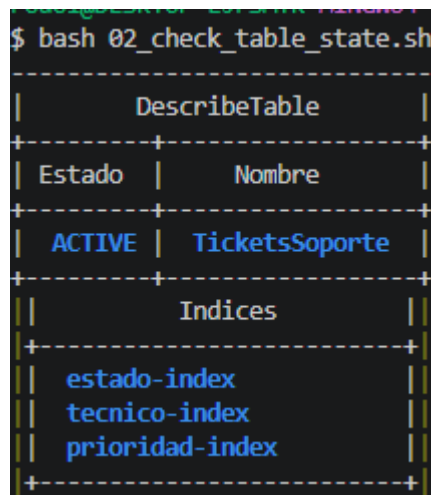
```
aws dynamodb create-table \  
  --table-name TicketsSoporte \  
  --attribute-definitions \  
    AttributeName=ticketId,AttributeType=S \  
    AttributeName=estado,AttributeType=S \  
    AttributeName=tecnico,AttributeType=S \  
    AttributeName=prioridad,AttributeType=S \  
  --key-schema AttributeName=ticketId,KeyType=HASH \  
  --billing-mode PAY_PER_REQUEST \  
  --global-secondary-indexes file://gsi.json \  
  --endpoint-url http://localhost:4566
```

Ejecución:

```
bash 01_create_table.sh
```

6.3 Verificar estado de la tabla (02_check_table_state.sh)

```
aws dynamodb describe-table \  
  --table-name TicketsSoporte \  
  --query  
"Table.{Nombre:TableName,Estado:TableStatus,Indices:GlobalSecondaryIndexes[*].Ind  
exName}" \  
  --output table \  
  --endpoint-url http://localhost:4566
```



```
$ bash 02_check_table_state.sh
```

DescribeTable	
Estado	Nombre
ACTIVE	TicketsSoporte

Indices	
estado-index	
tecnico-index	
prioridad-index	

Figura 15: verificacion de estado de tabla.

7. Datos de ejemplo insertados

Se insertaron 5 tickets representativos de los departamentos de Ventas, Finanzas y Soporte de ACME, cada uno en su propio archivo JSON (tk1.json a tk5.json), ejecutados mediante el script 03_insert_tickets.sh:

```
aws dynamodb put-item --table-name TicketsSoporte --item file://tk1.json
--endpoint-url http://localhost:4566
aws dynamodb put-item --table-name TicketsSoporte --item file://tk2.json
--endpoint-url http://localhost:4566
aws dynamodb put-item --table-name TicketsSoporte --item file://tk3.json
--endpoint-url http://localhost:4566
aws dynamodb put-item --table-name TicketsSoporte --item file://tk4.json
--endpoint-url http://localhost:4566
aws dynamodb put-item --table-name TicketsSoporte --item file://tk5.json
--endpoint-url http://localhost:4566
```

Ticket ID	Departamento	Estado	Prioridad	Técnico	Cliente
TK-2026-0001	Ventas	Abierto	Alta	Carlos Pérez	CL-001
TK-2026-0002	Finanzas	En proceso	Media	Ana Soto	CL-002
TK-2026-0003	Soporte	Cerrado	Baja	Luis Rojas	CL-003
TK-2026-0004	Ventas	Abierto	Crítica	Carlos Pérez	CL-001
TK-2026-0005	Finanzas	Abierto	Alta	María López	CL-004

Comandos de inserción de datos

```
aws dynamodb put-item --table-name TicketsSoporte --item file://tk1.json --endpoint-url http://localhost:4566
aws dynamodb put-item --table-name TicketsSoporte --item file://tk2.json
--endpoint-url http://localhost:4566
aws dynamodb put-item --table-name TicketsSoporte --item file://tk3.json
--endpoint-url http://localhost:4566
aws dynamodb put-item --table-name TicketsSoporte --item file://tk4.json
--endpoint-url http://localhost:4566
aws dynamodb put-item --table-name TicketsSoporte --item file://tk5.json
--endpoint-url http://localhost:4566
```

8. Consultas por índice secundario

8.1 Consulta 1 – Tickets Abiertos (05_query_by_estado.sh)

```
aws dynamodb query \
  --table-name TicketsSoporte \
  --index-name estado-index \
  --key-condition-expression "estado = :e" \
  --expression-attribute-values '{ ":e": {"S": "Abierto"} }' \
  --endpoint-url http://localhost:4566
```

Resultado: 3 tickets — TK-2026-0001, TK-2026-0004, TK-2026-0005 (todos con estado "Abierto")

```

{
  "Items": [
    {
      "ticketId": {
        "S": "TK-2026-0001"
      },
      "clienteId": {
        "S": "CL-001"
      },
      "departamento": {
        "S": "Ventas"
      },
      "categoria": {
        "S": "Software"
      },
      "prioridad": {
        "S": "Alta"
      },
      "estado": {
        "S": "Abierto"
      },
      "tecnico": {
        "S": "Carlos Perez"
      },
      "fechaCreacion": {
        "S": "2026-06-28"
      },
      "descripcion": {
        "S": "Error al iniciar sesion en el CRM"
      }
    },
    {
      "ticketId": {
        "S": "TK-2026-0004"
      },
      "clienteId": {
        "S": "CL-001"
      },
      "departamento": {
        "S": "Ventas"
      },
      "categoria": {
        "S": "Software"
      },
      "prioridad": {
        "S": "Critica"
      },
      "estado": {
        "S": "Abierto"
      },
      "tecnico": {
        "S": "Carlos Perez"
      },
      "fechaCreacion": {
        "S": "2026-06-28"
      },
      "descripcion": {
        "S": "Sistema de ventas caido durante presentacion"
      }
    },
    {
      "ticketId": {
        "S": "TK-2026-0005"
      },
      "clienteId": {
        "S": "CL-001"
      },
      "departamento": {
        "S": "Finanzas"
      },
      "categoria": {
        "S": "Software"
      },
      "prioridad": {
        "S": "Alta"
      },
      "estado": {
        "S": "Abierto"
      },
      "tecnico": {
        "S": "Maria Lopez"
      },
      "fechaCreacion": {
        "S": "2026-06-28"
      },
      "descripcion": {
        "S": "No se pudo generar reporte mensual"
      }
    }
  ],
  "Count": 3,
  "ScannedCount": 3,
  "ConsumedCapacity": null
}

```

Figura 16: tickets abiertos.

8.2 Consulta 2 – Tickets por técnico (06_query_by_tecnico.sh)

```

aws dynamodb query \
  --table-name TicketsSoporte \
  --index-name tecnico-index \
  --key-condition-expression "tecnico = :t" \
  --expression-attribute-values '{ ":t": {"S": "Carlos Perez"} }' \
  --endpoint-url http://localhost:4566

```

```

{
  "Items": [
    {
      "ticketId": {
        "S": "TK-2026-0001"
      },
      "clienteId": {
        "S": "CL-001"
      },
      "departamento": {
        "S": "Ventas"
      },
      "categoria": {
        "S": "Software"
      },
      "prioridad": {
        "S": "Alta"
      },
      "estado": {
        "S": "Abierto"
      },
      "tecnico": {
        "S": "Carlos Perez"
      },
      "fechaCreacion": {
        "S": "2026-06-28"
      },
      "descripcion": {
        "S": "Error al iniciar sesion en el CRM"
      }
    },
    {
      "ticketId": {
        "S": "TK-2026-0004"
      },
      "clienteId": {
        "S": "CL-001"
      },
      "departamento": {
        "S": "Ventas"
      },
      "categoria": {
        "S": "Software"
      },
      "prioridad": {
        "S": "Critica"
      },
      "estado": {
        "S": "Abierto"
      },
      "tecnico": {
        "S": "Carlos Perez"
      },
      "fechaCreacion": {
        "S": "2026-06-28"
      },
      "descripcion": {
        "S": "Sistema de ventas caido durante presentacion"
      }
    }
  ],
  "Count": 2,
  "ScannedCount": 2,
  "ConsumedCapacity": null
}

```

Resultado: 2 tickets asignados a Carlos Perez — TK-2026-0001, TK-2026-0004

8.3 Consulta 3 – Tickets por prioridad (07_query_by_prioridad.sh)

```
aws dynamodb query \  
  --table-name TicketsSoporte \  
  --index-name prioridad-index \  
  --key-condition-expression "prioridad = :p" \  
  --expression-attribute-values '{ ":p": {"S": "Critica"} }' \  
  --endpoint-url http://localhost:4566
```

```
{  
  "Items": [  
    {  
      "ticketId": {  
        "S": "TK-2026-0004"  
      },  
      "clienteId": {  
        "S": "CL-001"  
      },  
      "departamento": {  
        "S": "Ventas"  
      },  
      "categoria": {  
        "S": "Software"  
      },  
      "prioridad": {  
        "S": "Critica"  
      },  
      "estado": {  
        "S": "Abierto"  
      },  
      "tecnico": {  
        "S": "Carlos Perez"  
      },  
      "fechaCreacion": {  
        "S": "2026-06-28"  
      },  
      "descripcion": {  
        "S": "Sistema de ventas caido durante presentacion"  
      }  
    }  
  ],  
  "Count": 1,  
  "ScannedCount": 1,  
  "ConsumedCapacity": null  
}
```

Resultado: 1 ticket con prioridad Crítica — TK-2026-0004

9. Estrategia de respaldo

Backup local con Floci (08_create_backup.sh)

Como Floci opera en memoria, los datos no persisten entre reinicios. La estrategia de respaldo exporta todos los ítems de la tabla a un archivo JSON con marca de tiempo:

```
mkdir -p backups  
  
aws dynamodb scan \  
  --table-name TicketsSoporte \  
  --endpoint-url http://localhost:4566 \  
  --output json > backups/tickets_backup_$(date +%F_%H%M).json  
  
echo "Backup generado en backups/"
```

Servicios de cómputo

Implementación de una Aplicación Web en AWS EC2

Desplegar una aplicación web utilizando recursos de cómputo escalables mediante servicios compatibles con AWS EC2, implementados localmente con Floci.io para simular una infraestructura cloud, permitiendo administrar instancias, configurar servicios web y ejecutar aplicaciones de forma flexible, segura y escalable.

1. Instalación de Docker

Verificar instalación:

```
docker --version
```

Si Docker no está instalado:

Descargar desde: <https://www.docker.com/products/docker-desktop/>

2. Crear el archivo docker-compose.yml

Crear una carpeta del proyecto:

```
mkdir floci-project  
cd floci-project
```

Crear archivo:

```
nano docker-compose.yml
```

Contenido:

```
services:  
  floci:  
    image: floci/floci:latest  
    ports:  
      - "4566:4566"  
    volumes:  
      - ./data:/app/data
```

Guardar archivo.

3. Iniciar Floci

Ejecutar:

```
docker compose up -d
```

Verificar contenedor:

```
docker ps
```

Floci quedará disponible en:

<http://localhost:4566>

4. Configurar AWS CLI

Instalar AWS CLI:

```
aws --version
```

Configurar credenciales ficticias:

```
aws configure
```

Ingresar:

```
AWS Access Key ID: test
AWS Secret Access Key: test
Region: us-east-1
Output format: json
```

5. Crear una instancia EC2 simulada

Ejecutar:

```
aws --endpoint-url=http://localhost:4566 ec2 run-instances \
    --image-id ami-12345678 \
    --instance-type t2.micro
```

Listar instancias:

```
aws --endpoint-url=http://localhost:4566 ec2 describe-instances
```

6. Instalación del Servidor Web

Ingresar al contenedor EC2 simulado.

Actualizar paquetes:

```
sudo apt update
```

Instalar Nginx:

```
sudo apt install nginx -y
```

Verificar servicio:

```
sudo systemctl status nginx
```

7. Despliegue de la Aplicación Web

Copiar archivos:

```
scp -r proyecto usuario@localhost:/home/ubuntu/
```

Instalar dependencias:

```
npm install
```

Ejecutar aplicación:

```
npm start
```

8. Configuración de Proxy Reverso

Editar configuración de Nginx:

```
sudo nano /etc/nginx/sites-available/default
```

Agregar:

```
server {  
    listen 80;  
    location / {  
        proxy_pass http://localhost:3000;  
    }  
}
```

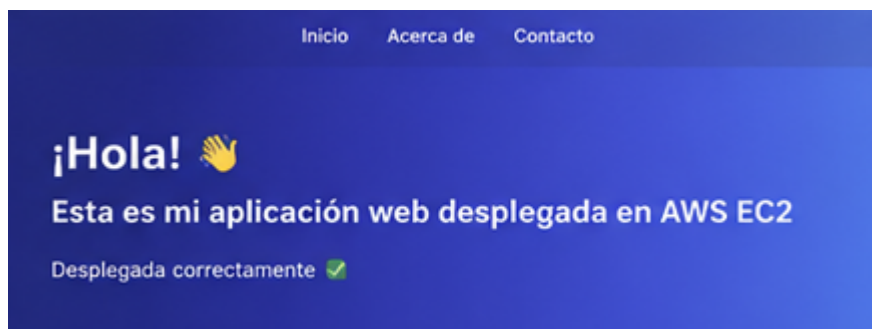
Reiniciar servicio:

```
sudo systemctl restart nginx
```

9. Verificación

Abrir navegador:

<http://localhost>



La aplicación web se visualiza correctamente.

10. Beneficios de Usar Floci

- No requiere cuenta AWS.
- No genera costos.
- Compatible con AWS CLI y SDK.
- Permite pruebas locales y CI/CD.

- Emula múltiples servicios AWS.
- Fácil integración con Docker.

5. Servicio de red en la nube

Una sola VPC con un tier público (expuesto a Internet, donde vive el balanceador) y un tier privado (donde vive la aplicación / base de datos, sin acceso directo desde Internet). El balanceador recibe el tráfico web y lo reparte hacia el backend privado.

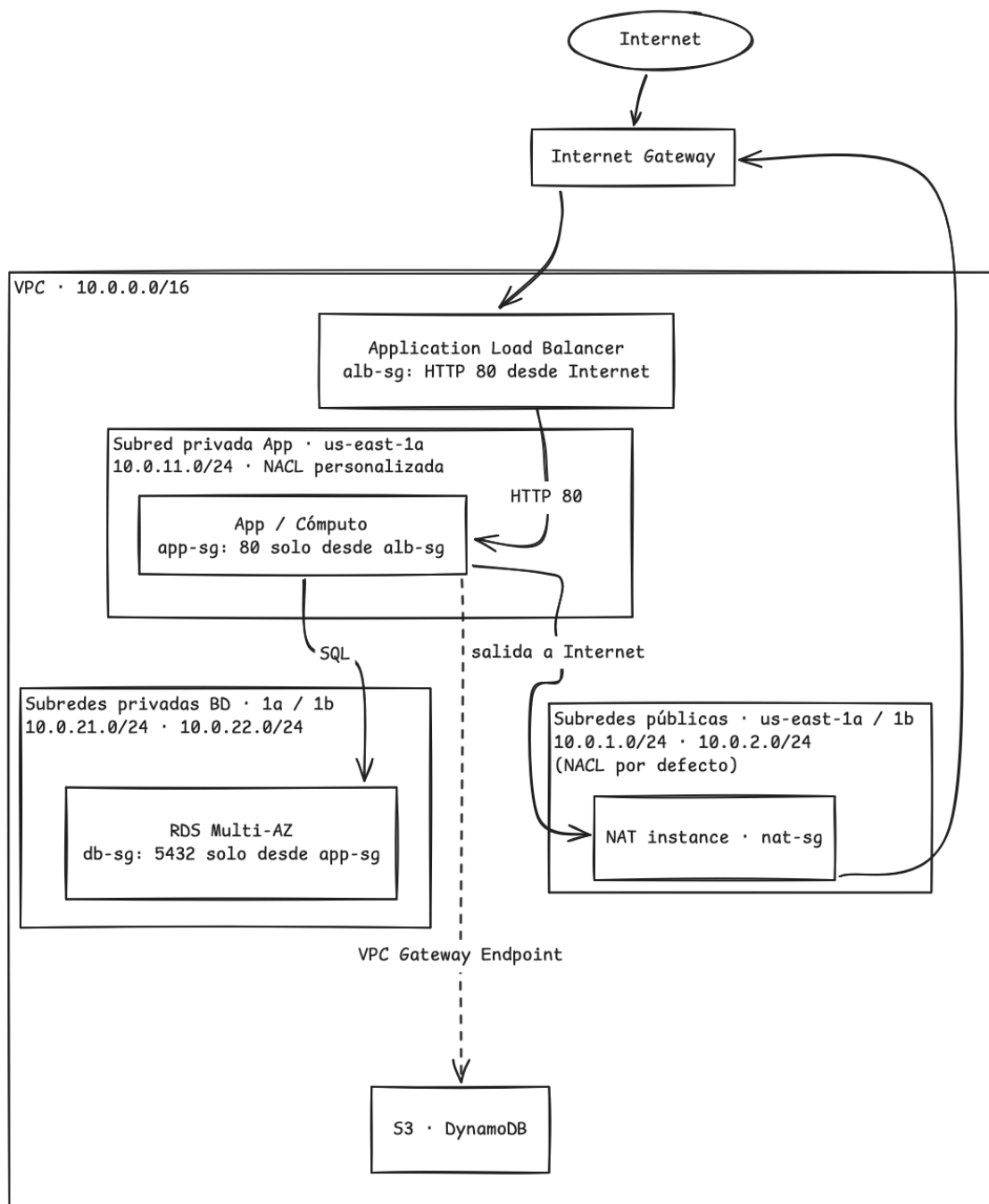


Figura 17: Diagrama de servicio de red para Acme.

Este diseño cumple con:

- Acceso externo restringido: El **ALB** mediante el puerto 80 es el único punto de entrada público, protegiendo los servicios internos de cualquier conexión directa desde Internet.
- Aislamiento en subredes **privadas**: Las aplicaciones y bases de datos operan en entornos aislados. Mientras la app gestiona actualizaciones vía **NAT Gateway**, la base de datos permanece totalmente desconectada del tráfico exterior.
- Defensa en profundidad: Se integran **Security Groups** para un control *stateful* por recurso y **NACL** como filtrado *stateless* a nivel de red.
- **Conectividad privada vía Endpoints**: La comunicación con S3 y DynamoDB se realiza a través de la infraestructura interna de AWS, **omitiendo el uso de Internet o NAT**, optimizando la seguridad y reduciendo costos operativos.

Descripción de componentes

Componente	Para qué sirve aquí	Tipo
VPC	Red aislada propia con rango IP privado	Red base
Subred pública	Aloja el ALB y el NAT; tiene ruta al IGW	Segmentación
Subred privada	Cómputo sin acceso público directo	Segmentación
Internet Gateway (IGW)	Entrada/salida a Internet de las subredes públicas	Conectividad
NAT Gateway	Permite salida a Internet de la subred privada sin exponerla	Conectividad
Route Tables	Definen quién enruta al IGW (pública) y quién al NAT (privada). Pública → IGW; privada → NAT	Enrutamiento
Security Groups	Firewall stateful por recurso	Seguridad
NACL	Filtro stateless por subred	Seguridad
VPC Gateway Endpoints	Acceso privado a S3 y DynamoDB	Conectividad
Application Load Balancer	Distribuye el tráfico HTTP hacia los targets	Balanceo

VPN Site-to-Site	<i>No aplica</i>	Conectividad híbrida
-------------------------	------------------	----------------------

El servicio de **VPN Site-to-Site** permite establecer un túnel IPSec seguro entre una infraestructura local y la VPC. No obstante, dado que el proyecto plantea una transición íntegra hacia entornos cloud, se prescinde de su implementación al no existir un centro de datos físico que requiera conexión constante. Esta tecnología sería pertinente únicamente si la organización conservase aplicaciones legadas on-premise que demanden comunicación privada con la nube.

Reglas de seguridad

#	Capa	Recurso	Regla
1	Security Group	alb-sg (entrada)	TCP 80 desde 0.0.0.0/0 (Internet → ALB)
2	Security Group	app-sg (entrada)	TCP 80 desde alb-sg (solo el ALB llega a la app)
3	NACL privada	entrada	ALLOW TCP 80 desde 10.0.0.0/16
4	NACL privada	salida	ALLOW TCP 1024-65535 hacia 10.0.0.0/16 (retorno <i>stateless</i>)

Implementación en AWS (cli)

Revisar implementación con scripts en repositorio:

- [Implementación de la red](#)
- [Limpieza de implementación](#)

El script implementa una VPC estructurada con niveles de seguridad diferenciados (público, aplicación privada y base de datos aislada), asegurando la conectividad de la capa privada mediante una instancia NAT. La arquitectura optimiza el tráfico hacia S3 y DynamoDB mediante el uso de Endpoints, sienta las bases para una base de datos RDS con configuración Multi-AZ y publica el servicio a través de un Load Balancer. Este diseño constituye el núcleo de red fundamental sobre el cual se desplegarán las instancias EC2 y los servicios gestionados de ACME.

Notificación y mensajería

Primero se creó un tema SNS llamado acme-alertas-topic, el cual funciona como canal central para publicar eventos o alertas relevantes de la infraestructura. Este tema representa el punto desde donde se distribuyen notificaciones hacia uno o más suscriptores.

```
● jfarias@jfarias:~/ARQUITECTURACLOUD/portafolio3$ TOPIC_ARN=$(aws sns create-topic \
--name acme-alertas-topic \
--query 'TopicArn' \
--output text)
● jfarias@jfarias:~/ARQUITECTURACLOUD/portafolio3$ echo $TOPIC_ARN
arn:aws:sns:us-east-1:000000000000:acme-alertas-topic
```

Figura 18: tema SNS creado.

Luego se creó una cola SQS llamada acme-alertas-queue, utilizada para recibir los mensajes enviados desde SNS. Esta cola permite que los mensajes queden disponibles hasta que otro servicio los procese, evitando una dependencia directa entre el sistema que genera la alerta y el sistema que la consume.

```
● jfarias@jfarias:~/ARQUITECTURACLOUD/portafolio3$ QUEUE_URL=$(aws sqs create-queue \
--queue-name acme-alertas-queue \
--attributes VisibilityTimeout=5,MessageRetentionPeriod=345600 \
--query 'QueueUrl' \
--output text)
● jfarias@jfarias:~/ARQUITECTURACLOUD/portafolio3$ echo $QUEUE_URL
http://localhost:4566/000000000000/acme-alertas-queue
```

Figura 19: cola SQS creada.

Posteriormente, se creó una cola adicional llamada acme-alertas-dlq, utilizada como Dead Letter Queue. Esta cola sirve para almacenar mensajes que no pudieron ser procesados correctamente después de varios intentos. Con esto se implementa un patrón básico de reintentos y manejo de fallos.

```
● jfarias@jfarias:~/ARQUITECTURACLOUD/portafolio3$ DLQ_URL=$(aws sqs create-queue \
--queue-name acme-alertas-dlq \
--query 'QueueUrl' \
--output text)

DLQ_ARN=$(aws sqs get-queue-attributes \
--queue-url "$DLQ_URL" \
--attribute-names QueueArn \
--query 'Attributes.QueueArn' \
--output text)

echo $DLQ_URL
echo $DLQ_ARN
http://localhost:4566/000000000000/acme-alertas-dlq
arn:aws:sqs:us-east-1:000000000000:acme-alertas-dlq
```

Figura 20: DLQ creada.

Para conectar SNS con SQS, se aplicó una política de permisos sobre la cola principal, permitiendo que el tema SNS pudiera enviar mensajes hacia ella. Luego se creó la suscripción entre acme-alertas-topic y acme-alertas-queue, logrando que cada mensaje publicado en SNS fuera recibido automáticamente por SQS.

```
jfarias@jfarias:~/ARQUITECTURACLOUD/portafolio3/mensajeria$ SUBSCRIPTION_ARN=$(aws sns subscribe \
--topic-arn "$TOPIC_ARN" \
--protocol sqs \
--notification-endpoint "$QUEUE_ARN" \
--query 'SubscriptionArn' \
--output text)

echo $SUBSCRIPTION_ARN
arn:aws:sns:us-east-1:000000000000:acme-alertas-topic:77530809-60fa-4f87-a15b-fbc2fb1dbeab
jfarias@jfarias:~/ARQUITECTURACLOUD/portafolio3/mensajeria$ aws sns list-subscriptions
{
  "Subscriptions": [
    {
      "SubscriptionArn": "arn:aws:sns:us-east-1:000000000000:acme-alertas-topic:77530809-60fa-4f87-a15b-fbc2fb1dbeab",
      "Owner": "000000000000",
      "Protocol": "sqs",
      "Endpoint": "arn:aws:sqs:us-east-1:000000000000:acme-alertas-queue",
      "TopicArn": "arn:aws:sns:us-east-1:000000000000:acme-alertas-topic"
    }
  ]
}
```

Figura 21: suscripción SNS hacia SQS.

Como prueba funcional, se publicó una alerta simulada de CPU alta desde SNS. El mensaje incluía información del servicio afectado, tipo de alerta, severidad y acción sugerida. Luego, el mensaje fue leído desde la cola SQS, comprobando que el flujo de mensajería funcionaba correctamente.

```
jfarias@jfarias:~/ARQUITECTURACLOUD/portafolio3/mensajeria$ aws sns publish --topic-arn "$TOPIC_ARN" --subject "Alerta CPU Alta"
--message '{"servicio":"api-acme","tipo":"CPU_ALTA","valor":"85%","severidad":"alta","accion":"revisar escalabilidad"}'
{
  "MessageId": "ac3cc85e-4350-42d1-8824-94a429fcf0c7"
}
```

Figura 22: mensaje publicado en SNS.

```
jfarias@jfarias:~/ARQUITECTURACLOUD/portafolio3/mensajeria$ aws sqs receive-message \
--queue-url "$QUEUE_URL" \
--max-number-of-messages 10 \
--attribute-names All \
--message-attribute-names All \
--wait-time-seconds 2 \
--output json
{
  "Messages": [
    {
      "MessageId": "bda491bb-e35f-42c0-a003-4807fec7d3b5",
      "ReceiptHandle": "431a42e5-df1b-4024-8bf8-1fd04bbae0c7",
      "MD5OfBody": "cef8dc0563c79667ff9d70be1262cde",
      "Body": "{\"Type\":\"Notification\",\"MessageId\":\"ac3cc85e-4350-42d1-8824-94a429fcf0c7\",\"TopicArn\":\"arn:aws:sns:us-east-1:000000000000:acme-alertas-topic\",\"Timestamp\":\"2026-06-29T06:34:28.073957935Z\",\"Subject\":\"Alerta CPU Alta\",\"Message\":{\"servicio\":\"api-acme\",\"tipo\":\"CPU_ALTA\",\"valor\":\"85%\",\"severidad\":\"alta\",\"accion\":\"revisar escalabilidad\"}}\", \"MessageAttributes\": {}}",
      "Attributes": {
        "SenderId": "000000000000",
        "SentTimestamp": "1782714868074",
        "ApproximateReceiveCount": "1",
        "ApproximateFirstReceiveTimestamp": "1782714888286"
      }
    }
  ]
}
```

Figura 23: mensaje recibido desde SQS.

También se configuró una política de reintentos mediante RedrivePolicy, definiendo que los mensajes fallidos fueran enviados a la DLQ después de tres intentos. Esto permite mejorar la tolerancia a fallos, ya que los mensajes no procesados no se pierden, sino que quedan disponibles para revisión posterior.


```
jfarias@jfarias:~/ARQUITECTURACLOUD/portafolio3/mensajeria$ aws --endpoint-url=http://localhost:4566 sqs get-queue-attributes \
--queue-url "$QUEUE_URL" \
--attribute-names RedrivePolicy \
--output json | jq
{
  "Attributes": {
    "RedrivePolicy": "{\"deadLetterTargetArn\":\"arn:aws:sqs:us-east-1:000000000000:acme-alertas-dlq\",\"maxReceiveCount\":\"3\"}"
  }
}
```

Figura 24: política RedrivePolicy configurada.

Respecto a la integración con correo o servicios externos, en esta demo local se utilizó SQS como canal de mensajería funcional. En un ambiente real de AWS, el mismo tema SNS podría integrarse con correo electrónico, SMS, Lambda o endpoints HTTP/S para enviar alertas automáticas al equipo de soporte.

Servicios simples de alojamiento web y contenidos

En esta etapa, se necesita poder crear un bucket S3 donde se alojará el sitio web. Pero antes de comenzar, Floci debe estar corriendo y la AWS CLI debe apuntar a `http://localhost:4566`.

Para que no haya problemas en la ejecución de comandos, se debe configurar AWS CLI

```
PS C:\Users\usuario> $env:AWS_ENDPOINT_URL="http://localhost:4566"
PS C:\Users\usuario> $env:AWS_ACCESS_KEY_ID="test"
PS C:\Users\usuario> $env:AWS_SECRET_ACCESS_KEY="test"
PS C:\Users\usuario> $env:AWS_DEFAULT_REGION="us-east-1"
```

Pasos a realizar:

1. Crear un bucket S3 llamado "leccion_7":

```
PS C:\Users\usuario> aws s3 mb s3://leccion_7 --endpoint-url=http://localhost:4566
make_bucket: leccion_7
```

2. Crear carpeta "sitio_web", la cual alojará el archivo HTML:

```
PS C:\Users\usuario> aws s3api put-object --bucket leccion_7 --key sitio_web/
{
  "ETag": "\"d41d8cd98f00b204e9800998ecf8427e\"",
  "ChecksumCRC64NVME": "AAAAAAAAAAAA="
}
```

3. Subir archivo "index.html":

```
PS C:\Users\usuario> aws s3 cp index.html s3://leccion_7/sitio_web/index.html --endpoint-url http://localhost:4566
upload: .\index.html to s3://leccion_7/sitio_web/index.html
```

Validar que se haya subido bien el archivo

```
PS C:\Users\usuario\OneDrive\Desktop\Arquitectura Cloud\Portafolio 4> aws s3 ls s3://leccion_7/sitio_web/ --endpoint-url http://localhost:4566
2026-06-30 17:25:27      0
2026-06-30 17:54:49    692 index.html
```

Validar que la página web se vea



Figura 25: página web simple.

4. Debido a que no existe Cloud Front como tal en Floci, se debe crear un archivo json para configurar distribución:

```

{
  "CallerReference": "test-distribution-1",
  "Comment": "Mi CDN local en Floci",
  "Enabled": true,
  "Origins": {
    "Quantity": 1,
    "Items": [
      {
        "Id": "S3-Leccion_7",
        "DomainName": "leccion_7.localhost.localstack.cloud",
        "S3OriginConfig": {
          "OriginAccessIdentity": ""
        }
      }
    ]
  },
  "DefaultCacheBehavior": {
    "TargetOriginId": "S3-Leccion_7",
    "ViewerProtocolPolicy": "allow-all",
    "TrustedSigners": {
      "Quantity": 0,
      "Enabled": false
    },
    "ForwardedValues": {
      "QueryString": false,
      "Cookies": {
        "Forward": "none"
      }
    },
    "Headers": {
      "Quantity": 0
    },
    "QueryStringCacheKeys": {
      "Quantity": 0
    }
  },
  "MinTTL": 0
}

```

5. Ejecutar json y validar distribución:

```

PS C:\Users\usuario\OneDrive\Desktop\Arquitectura Cloud\Portafolio 4> aws cloudfront create-distribution --distribution-config file://dist-config.json --endpoint-url http://localhost:4566
{
  "Location": "http://localhost:4566/2020-05-31/distribution/E12FHXA0ABMSABH",
  "ETag": "453a0251-0d94-4299-90c3-b0b0b48ec96c",
  "Distribution": {
    "Id": "E12FHXA0ABMSABH",
    "ARN": "arn:aws:cloudfront::000000000000:distribution/E12FHXA0ABMSABH",
    "Status": "Deployed",
    "LastModifiedTime": "2020-06-30T22:21:35.817692+00:00",
    "DomainName": "E12FHXA0ABMSABH.cloudfront.net",
    "DistributionConfig": {
      "CallerReference": "test-distribution-1",
      "Aliases": {
        "Quantity": 0
      },
      "DefaultRootObject": "",
      "Origins": {
        "Quantity": 1,
        "Items": [
          {
            "Id": "S3-MiBucketOrigen",
            "DomainName": "mi-bucket-estatico.s3.localhost.localstack.cloud",
            "OriginPath": "",
            "ViewerProtocolPolicy": "allow-all",
            "S3OriginConfig": {
              "OriginAccessIdentity": ""
            },
            "ConnectionAttempts": 3,
            "ConnectionTimeout": 10
          }
        ]
      },
      "DefaultCacheBehavior": {
        "TargetOriginId": "S3-MiBucketOrigen",
        "ViewerProtocolPolicy": "allow-all",
        "AllowedMethods": {
          "Quantity": 2,
          "Items": [
            "GET",
            "HEAD"
          ]
        },
        "Compress": false,
        "LambdaFunctionAssociations": {
          "Quantity": 0
        },
        "FunctionAssociations": {
          "Quantity": 0
        }
      },
      "CacheBehaviors": {
        "Quantity": 0
      },
      "CustomErrorResponses": {
        "Quantity": 0
      },
      "Comment": "Mi CDN local en Floci",
      "PriceClass": "PriceClass_All",
      "Enabled": true,
      "ViewerCertificate": {
        "CloudFrontDefaultCertificate": true,
        "MinimumProtocolVersion": "TLSv1.2_2021"
      },
      "HttpVersion": "http2",
      "IsIPV6Enabled": true
    }
  }
}

```

6. Configurar certificados

Como no es posible realizarlo directamente en floci, se debe pasar por nginx y utilizarlo como puente.

a. Generar archivo nginx.conf

```

events { worker_connections 1024; }

http {
    server {
        listen 4433 ssl;
        server_name localhost;

        ssl_certificate /etc/nginx/certs/server.crt;
        ssl_certificate_key /etc/nginx/certs/server.key;

        ssl_protocols TLSv1.2 TLSv1.3;
        ssl_ciphers HIGH:!aNULL:!MD5;

        location / {
            proxy_pass http://floci-backend:4566;
            proxy_set_header Host $host;
            proxy_set_header X-Real-IP $remote_addr;
            proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
            proxy_set_header X-Forwarded-Proto $scheme;
            client_max_body_size 0;
        }
    }
}

```

Se debe generar los certificados con el siguiente comando

```
PS C:\Users\usuario\OneDrive\Desktop\Arquitectura Cloud\Portafolio 4\nginx-1.31.2\conf> docker run --rm -v "${PWD}:/export" alpine sh -c "apk add --no-cache openssl && openssl req -x509 -newkey rsa:4096 -keyout /export/server.key -out /export/server.crt -days 365 -nodes -subj '/CN=localhost'"
Unable to find image 'alpine:latest' locally
latest: Pulling from library/alpine
55afalecc21d: Pull complete
56dceff1b33: Download complete
f5124fb579e2: Download complete
Digest: sha256:28bd5fe8b56d1bd048e8babf5b10710ebe0bae67db86916198a6ec434943f8b
Status: Downloaded newer image for alpine:latest
(1/1) Installing openssl (3.5.7-r9)
Executing busybox-1.37.0-r31.trigger
OK: 8997 KiB in 17 packages

PS C:\Users\usuario\OneDrive\Desktop\Arquitectura Cloud\Portafolio 4\nginx-1.31.2\conf> Get-ChildItem server.*

Directorio: C:\Users\usuario\OneDrive\Desktop\Arquitectura Cloud\Portafolio 4\nginx-1.31.2\conf

Mode                LastWriteTime         Length Name
----
-a---l             30-06-2026    23:06         1895 server.crt
-a---l             30-06-2026    23:06         3272 server.key
```

Esto generará 2 archivos en la ruta donde está nginx.conf, los cuales corresponden a los certificados:

 server.crt

 server.key

b. Como floci no permite configurar un SSL en un bucket ya creado, se debe eliminar cualquier contenedor o red vieja que se llame igual

```
PS C:\Users\usuario\OneDrive\Desktop\Arquitectura Cloud\Portafolio 4\nginx-1.31.2\conf> docker rm -f floci floci-ssl floci-backend nginx-proxy floci
Error response from daemon: No such container: floci-ssl
Error response from daemon: No such container: floci-backend
Error response from daemon: No such container: nginx-proxy
PS C:\Users\usuario\OneDrive\Desktop\Arquitectura Cloud\Portafolio 4\nginx-1.31.2\conf> docker network rm floci-network
Error response from daemon: network floci-network not found
exit status 1
```

c. Se crea la red virtual para que se comuniquen entre sí

```
PS C:\Users\usuario\OneDrive\Desktop\Arquitectura Cloud\Portafolio 4\nginx-1.31.2\conf> docker network create floci-network 022076b31c9527f0ad86da1cba54e360c5dda0b5bbb36d37d1171cc383f95b73
```

d. Encender Floci en su modo nativo estable

```
PS C:\Users\usuario\OneDrive\Desktop\Arquitectura Cloud\Portafolio 4\nginx-1.31.2\conf> docker run -d --name floci-backend --network floci-network -p "4566:4566" -v floci-data:/var/lib/floci floci/floci:latest
109c198d52fd037a2e869f51e21d1a0705b49e70b6db95599dc8f32280f25970
```

e. Encender Nginx, pasándole los certificados generados y la configuración

```
PS C:\Users\usuario\OneDrive\Desktop\Arquitectura Cloud\Portafolio 4\nginx-1.31.2\conf> docker run -d --name nginx-proxy --network floci-network -p "4433:4433" -v "${PWD}/server.crt:/etc/nginx/certs/server.crt:ro" -v "${PWD}/server.key:/etc/nginx/certs/server.key:ro" nginx:alpine
Unable to find image 'nginx:alpine' locally
alpine: Pulling from library/nginx
c7509c33e8b0: Pull complete
907885d218c5: Pull complete
e6f31ffc071e: Pull complete
c16defe09b2f: Pull complete
30420a4b30df: Pull complete
ab1fd98049751: Pull complete
ce42635eeddd: Pull complete
01bf363d61e6: Pull complete
9e60f8ae91af: Download complete
80fd3fc5237f: Download complete
Digest: sha256:54f2a904c251d5a34adf545a72d32515a15e00418dae0266e23be2e18c66fefa
Status: Downloaded newer image for nginx:alpine
49740becadb3a359b552bc0af3b09945d02780fcca0f37f4a7a1f2380f8faa2
```

f. Volver a ejecutar los pasos 1, 2 y 3 de esta sección indicados anteriormente

g. Validar



Figura 26: Validación de página web.

Conclusión

En conclusión, el proyecto “Infraestructura Viva” permitió diseñar y validar una propuesta integral de modernización cloud para Soluciones Digitales ACME. Mediante el uso de Floci, fue posible simular servicios similares a AWS y demostrar cómo una organización puede avanzar desde una infraestructura local hacia una solución más escalable, segura y flexible, sin depender inicialmente de recursos cloud pagados.

La propuesta integra almacenamiento, bases de datos, cómputo, red, monitoreo y mensajería, cubriendo los principales requerimientos del módulo. Además, se aplicaron buenas prácticas como separación de servicios, respaldo de datos, control de acceso, políticas de reintentos, monitoreo y documentación de costos estimados. En general, este portafolio evidencia cómo los conceptos de tecnología cloud pueden aplicarse en un caso práctico, permitiendo construir una base sólida para futuras implementaciones reales en AWS u otros proveedores de nube.